# CardioIA — Relatório Técnico da Fase 7

Coração Sob Controle: plataforma integrada de previsão de crises com IA

**Integrantes:** Michael Rodrigues · Arthur Alentejo · Nathalia Vasconcelos | **Tutor:** Caique | **Coordenador:** André Godói | FIAP, turma 2TIAO, 2026

| Entregável | Endereço |
|---|---|
| Aplicação Web (Vercel) | https://tiao-2026.vercel.app |
| APK Android (EAS Build) | expo.dev/accounts/microdri/projects/cardioia/builds/182ecc37-8deb-4059-8612-1fac1bcf6171 |
| API integradora (Render) | https://cardioia-backend-phkq.onrender.com |
| Simulação Wokwi | *link público do projeto (a adicionar)* |
| Vídeo demonstrativo | https://youtu.be/OtVBNq8AOJ8 |
| Código-fonte | github.com/Fiap-RJ/TIAO-2026, pasta `2TIAO/FASE7-CARDIOIA` (repositório privado, compartilhado com o tutor) |

## 1. Objetivo

As fases anteriores da CardioIA produziram peças independentes: coleta por sensores (Fase 3), visão computacional sobre ECG (Fase 4), um assistente conversacional no IBM Watson (Fase 5) e um modelo preditivo de picos de risco com agentes (Fase 6). A Fase 7 junta essas peças num produto funcional. Um dispositivo simulado mede o paciente, um backend em Python aplica os modelos de IA, e médico e paciente acompanham o resultado em tempo real pela Web e por um aplicativo Android.

Os requisitos atendidos são: (1) deploy da aplicação Web com CI/CD e distribuição do aplicativo como APK; (2) refinamento da experiência de uso dos indicadores de risco; (3) um backend Python que conecta as interfaces aos motores de IA das Fases 5 e 6; (4) conversão da lógica dos sensores de C/C++ para MicroPython, simulada no Wokwi.

## 2. Arquitetura final

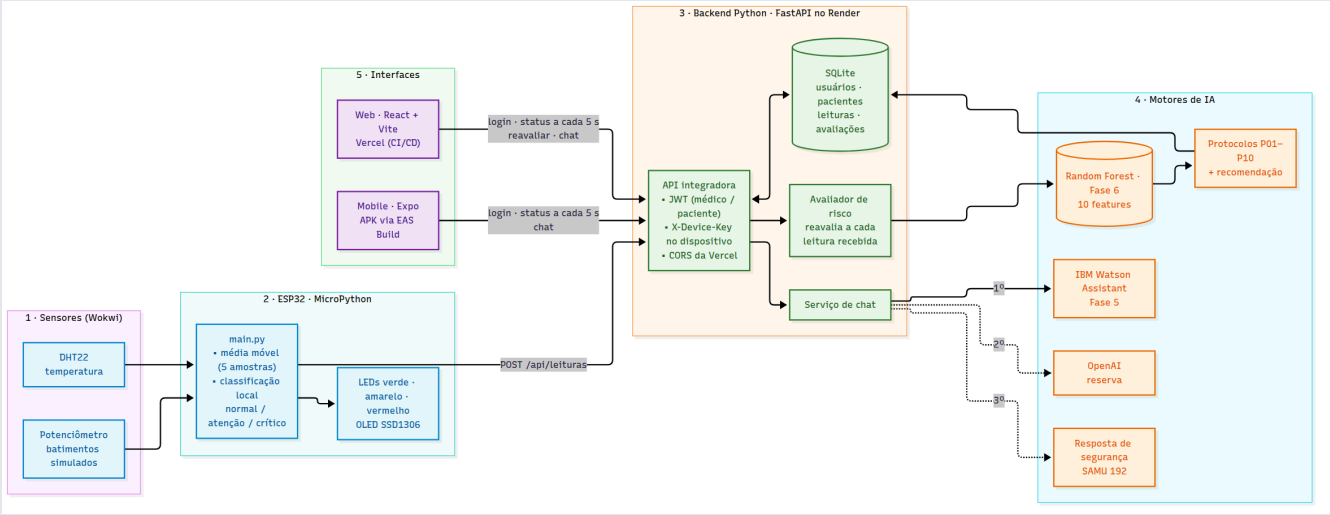



Figura 1 — Fluxo Sensor → MicroPython → Backend Python → APIs de IA → UI. Fonte: `document/fonte/arquitetura_final.mmd`.

A arquitetura tem cinco camadas, na ordem em que o dado percorre o sistema:

| Camada | Tecnologia | Responsabilidade |
|---|---|---|
| 1. Sensores | DHT22 e potenciômetro (Wokwi) | Temperatura e frequência cardíaca simulada. |
| 2. Borda | ESP32 com MicroPython | Filtra o sinal, classifica o estado localmente, dá feedback em LEDs e OLED e envia a leitura por HTTP. |
| 3. Backend | FastAPI, SQLite, Render | Autentica usuários e dispositivo, persiste leituras e avaliações, dispara a reavaliação de risco e atende o chat. |
| 4. Motores de IA | Random Forest (Fase 6), Watson (Fase 5), OpenAI | Probabilidade de pico de risco, protocolos P01–P10, ações operacionais e triagem conversacional. |
| 5. Interfaces | React + Vite (Vercel), Expo (APK) | Painel do médico e do paciente com risco, sinais vitais em tempo real, recomendação e chat. |

## 3. Fluxo de dados fim a fim

1. **Captura:** a cada 5 s o ESP32 lê o DHT22 e o potenciômetro (0–4095 → 40–180 bpm) e calcula a média móvel das últimas 5 leituras de frequência cardíaca.
2. **Decisão na borda:** o firmware classifica o paciente como *crítico* (FC < 50 ou > 120 bpm, ou temperatura > 38 °C), *atenção* (FC 50–59 ou 101–120 bpm, ou temperatura 37,5–38 °C) ou *normal*. Acende o LED correspondente e mostra FC, temperatura e estado no OLED, sem depender da rede.
3. **Envio:** `POST /api/leituras` com `paciente_id`, `fc`, `temperatura`, `spo2` e `estado`, autenticado pelo header `X-Device-Key`. Falhas de rede são tratadas sem travar o laço de leitura.
4. **Persistência e reavaliação:** o backend valida a chave e os limites fisiológicos, registra a leitura com o horário do servidor e recalcula o risco imediatamente, a cada leitura. Assim a classificação sobe numa crise e volta a baixar quando o sinal se normaliza.
5. **Inferência:** o avaliador monta as 10 features do modelo da Fase 6 com a média das 5 leituras mais recentes (FC, $SpO_2$, temperatura) e os indicadores operacionais simulados do paciente. O Random Forest devolve a probabilidade, a faixa de risco é aplicada e as regras P01–P10 selecionam protocolos, sinais de atenção e ações. O resultado é salvo no formato `RespostaAtendimento` da Fase 6.
6. **Entrega:** Web e APK consultam `GET /api/pacientes/{id}/status` a cada 5 s e atualizam o medidor de risco, os gráficos e a recomendação. O médico pode pedir uma reavaliação manual, e qualquer perfil pode conversar com o assistente por `POST /api/chat`.

## 4. Componentes

### 4.1 IoT: de C/C++ para MicroPython

O firmware da Fase 3 (Arduino/C++) lia os sensores e publicava valores brutos via MQTT, sem interface no dispositivo. A versão em MicroPython (`iot/main.py`) transforma o ESP32 em um nó de borda com decisão local.

| Aspecto | Fase 3 (C/C++) | Fase 7 (MicroPython) |
|---|---|---|
| Tratamento do sinal | `map()` direto do ADC | ADC com atenuação de 11 dB e média móvel de 5 amostras |
| Classificação | Inexistente | Normal, atenção e crítico no próprio dispositivo |
| Feedback | Monitor serial | 3 LEDs e display OLED SSD1306 (I2C, GPIO 21/22) |
| Comunicação | MQTT com broker externo | HTTP POST direto para a API, com `X-Device-Key` |
| Resiliência | Buffer em `String` | `try/except` no envio e liberação do socket |

A comparação completa está em `document/conversao_c_micropython.md`. Na simulação, o DHT22 inicia em 36,6 °C e a $SpO_2$ é enviada fixa em 97 %, já que o circuito não tem oxímetro.

### 4.2 Backend integrador (Python)

| Rota | Consumidor | Função |
|---|---|---|
| `POST /api/auth/login` | Web, APK | Login com JWT (HMAC-SHA256, 8 h) e perfil médico ou paciente; senhas com PBKDF2. |
| `POST /api/leituras` | ESP32 | Recebe a telemetria (chave obrigatória) e reavalia o risco a cada leitura. |
| `GET /api/pacientes` | Web, APK | Pacientes com última leitura e último risco; o paciente vê só o próprio cadastro. |
| `GET /api/pacientes/{id}/status` | Web, APK | Últimas 60 leituras e a avaliação mais recente. |
| `POST /api/pacientes/{id}/avaliar` | Web | Reavaliação manual, restrita ao perfil médico. |
| `POST /api/chat` | Web, APK | Assistente de triagem com sessão reaproveitada entre mensagens. |
| `GET /api/health` | Render | Verificação de saúde do serviço. |

O código da Fase 6 (modelo, domínio, protocolos e recomendação) foi copiado para `backend/cardioia/`, o que deixa o deploy da fase independente. Os testes automatizados cobrem o fluxo completo de telemetria, o isolamento entre pacientes e o contrato do chat sem provedores externos.

### 4.3 Motores de IA

- **Modelo preditivo (Fase 6):** Random Forest com 200 árvores sobre 10 indicadores. No conjunto de teste da Fase 6, acurácia de 0,999 e F1 de 0,998 para a classe de pico. Faixas: BAIXO (< 0,40), MODERADO (0,40–0,70) e ALTO (≥ 0,70).
- **Protocolos e recomendação (Fase 6):** as regras P01–P10 comparam cada indicador com o seu gatilho, apontam sinais de atenção (acima do limiar de risco, abaixo do gatilho) e geram ações operacionais com justificativa.
- **Chat (Fase 5):** o IBM Watson Assistant V2 faz a triagem e recria a sessão quando ela expira. Se o Watson falhar, o OpenAI responde com instruções que proíbem diagnóstico e prescrição; nesse caminho, perguntas sobre doses ou medicamentos são recusadas antes mesmo da chamada ao modelo. Sem nenhum provedor disponível, o sistema devolve uma orientação fixa de emergência (SAMU 192).

### 4.4 Interfaces Web e Mobile

As duas aplicações usam o mesmo contrato de API e as mesmas telas: login; lista de pacientes ordenada pelo risco, com borda colorida por faixa; painel com medidor semicircular da probabilidade (marcas em 0,40 e 0,70), sinais vitais com tendência e linhas de limiar, protocolos acionados, sinais de atenção, ações e explicação; e o chat. As decisões de experiência de uso foram:

- a cor nunca aparece sozinha: toda faixa e todo estado têm rótulo em texto;
- o aviso legal fica visível em todas as telas de dados;
- quando o backend demora (inicialização do Render gratuito), a interface avisa que o servidor está iniciando em vez de parecer travada;
- o paciente vê apenas o próprio painel, e ações clínicas como a reavaliação ficam restritas ao médico;
- sem backend configurado, os apps entram num modo de demonstração identificado na tela, útil para desenvolvimento e testes.

## 5. Deploy e distribuição

| Alvo | Configuração |
|---|---|
| Web (Vercel) | Projeto ligado ao GitHub com Root Directory `2TIAO/FASE7-CARDIOIA/web`. O `vercel.json` reescreve todas as rotas para `index.html`, o que permite abrir ou recarregar qualquer rota da SPA. Cada push na `main` gera um deploy de produção, e cada pull request gera uma URL de pré-visualização. A URL da API entra como `VITE_API_URL`. |
| Mobile (EAS) | `app.json` com `android.package` `br.com.fiap.cardioia` (domínio invertido); `eas.json` com o perfil `preview`, distribuição interna e `buildType: "apk"`. O build roda na nuvem do Expo, que gera e guarda a keystore de assinatura. A URL da API entra como `EXPO_PUBLIC_API_URL` no momento do build. |
| Backend (Render) | Serviço web Python definido em `render.yaml`, com health check em `/api/health` e deploy automático a partir da `main`. Segredos (`JWT_SECRET`, `DEVICE_KEY`, chaves do Watson e do OpenAI) ficam só no painel do Render. |
| Verificação | Workflow do GitHub Actions que, a cada push ou pull request, roda lint e build do Web e typecheck, lint, `expo-doctor` e geração do bundle Android do app. |

**Validação:** a aplicação Web foi testada na URL pública, incluindo o acesso direto a rotas internas, e o APK foi instalado em um aparelho Android real para validar o login e a visualização dos dados cardíacos. As evidências estão em `assets/prints/` e no vídeo demonstrativo.

## 6. Decisões de projeto

- **Avaliação em tempo real sem LLM.** O pipeline com agentes da Fase 6 leva alguns segundos e tem custo por chamada. Executá-lo a cada leitura do sensor, a cada 5 s, seria lento e caro. Por isso a reavaliação automática usa diretamente o modelo e as regras da Fase 6, que são os componentes que calculam os números também no pipeline com agentes. O LLM fica no chat, onde a conversa justifica a latência.
- **HTTP em vez de MQTT.** O envio direto para a API elimina o broker externo, deixa a autenticação do dispositivo no mesmo serviço e permite reavaliar o risco no mesmo request que grava a leitura.
- **Polling a cada 5 s.** Mais simples e estável que WebSocket no plano gratuito, e alinhado ao intervalo de envio do sensor.
- **Horário registrado no servidor.** O ESP32 simulado não tem relógio sincronizado, então o backend carimba cada leitura no recebimento.
- **Contrato único.** O formato da avaliação é o mesmo schema da Fase 6, e Web e APK compartilham o mesmo contrato (`document/contrato_api.md`), o que permitiu desenvolver as três partes em paralelo.

## 7. Limitações e próximos passos

- **Dados sintéticos:** o modelo foi treinado com dados sintéticos sem ruído, e os sinais vêm de componentes virtuais. Os resultados não têm validade clínica.
- **$SpO_2$ fixa:** o circuito não tem oxímetro. Um próximo passo é incluir um sensor simulado de $SpO_2$ (por exemplo, um segundo potenciômetro).
- **Persistência:** o SQLite no disco temporário do Render gratuito é recriado a cada reinício. Em produção, o caminho seria um banco gerenciado, como PostgreSQL.
- **Inicialização a frio:** o plano gratuito desliga o backend sem uso, e a primeira chamada pode levar até um minuto.
- **Segurança:** as contas de demonstração e a chave única por dispositivo atendem ao MVP. Em produção seriam necessários cadastro real, chave por dispositivo, rotação de segredos e trilha de auditoria.

> **Aviso:** o CardioIA é um sistema acadêmico, informativo e de apoio à decisão operacional, baseado em dados sintéticos. Não realiza diagnóstico, não prescreve tratamentos e não substitui a avaliação de um profissional de saúde.